LABORATORY PROJECT REPORT

SERIAL INTERFACING WITH MICROCONTROLLER: SENSORS AND ACTUATORS

EXPERIMENT 4A

Section 1

Group 2

Semester 2 2025/2026

NO	NAME	MATRIC NO
1	AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI	2310663
2	ALYA SYAFINA BINTI ABDUL RAZAK	2218084

DATE OF SUBMISSION: 1 APRIL 2026

Abstract

In this experiment, we developed a simple hand gesture recognition system using an MPU6050 motion sensor and an Arduino Uno. The MPU6050 collected acceleration and gyroscope data to detect the movement and orientation of the hand. This data was sent to the computer through serial communication, where a Python program processed and displayed it. By setting certain conditions, the system was able to recognize basic hand gestures and show their movement paths on an x-y graph. Through this experiment, we learned how to use I2C communication between the Arduino and MPU6050, and how to send real-time data to the computer for analysis. Overall, this experiment helped us understand the basic concept of motion sensing and how it can be used for gesture-based control systems.

Table of Contents

1.0 **Introduction**

2.0 **Materials and Equipment**

2.1 Electronic Components

2.2 Equipment and Tools

3.0 **Experimental Setup**

4.0 **Methodology**

5.0 **Data Collection**

6.0 **Data Analysis**

7.0 **Results**

8.0 **Discussion**

9.0 **Conclusion**

10.0 **Recommendations**

11.0 **References**

1.0 Introduction

This experiment focuses on developing a simple hand gesture recognition system using an MPU6050 motion sensor, an Arduino Uno, and a computer. The MPU6050 sensor contains a 3-axis accelerometer and a 3-axis gyroscope that can measure motion and orientation of the hand. The main purpose of this experiment is to understand how motion data can be collected, processed, and used to identify different hand gestures.

The MPU6050 communicates with the Arduino through the I2C protocol, which allows data to be sent efficiently with only two communication lines. The Arduino reads the real-time sensor data and sends it to a computer via serial communication. On the computer side, a Python program using the *pyserial* library receives and processes the data to visualize the motion and detect specific gestures.

Through this experiment, we learned how to interface a motion sensor with a microcontroller, use I2C communication, and transmit real-time data to a computer for analysis. This experiment also helps us understand the basic concept of inertial sensing and how motion sensors can be used in applications such as gesture control, robotics, and human–computer interaction.

2.0 Materials and Equipment

2.1 Electronic Components

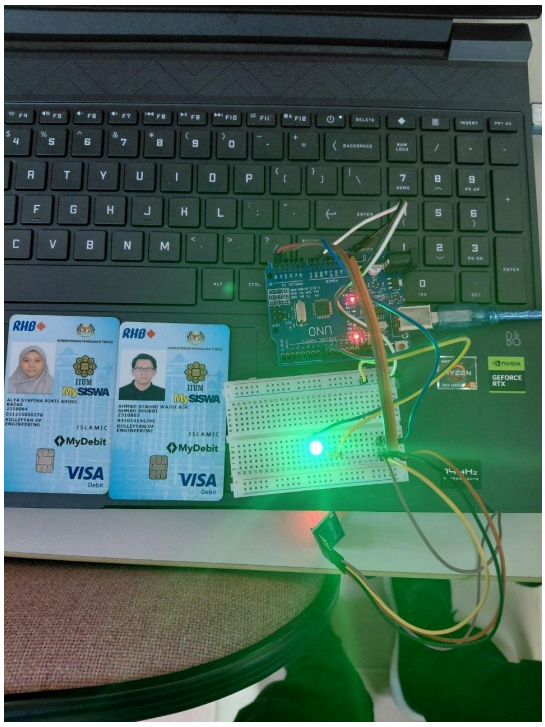
- Arduino Board – Microcontroller board for controlling the display
- MPU6050 Sensor - to read data such as detecting hand gestures
- Connecting Wires - establish connections between components
- Breadboard - to attach the MPU 6050

2.2 Equipment and Tools

- Arduino IDE – Software for writing and uploading code to the Arduino Uno
- USB cable – For connecting the Arduino Uno to a computer
- Power Supply (5V via USB or external source)
- Pycharm

3.0 <u>Experimental Setup</u>

1. The MPU6050 sensor was connected to the Arduino board using the appropriate pins. The MPU6050 typically used I2C communication, so the SDA and SCL pins of the MPU6050 were connected to the corresponding pins on the Arduino (usually A4 and A5 for most Arduino boards).
2. The power supply and ground of the MPU6050 were connected to the Arduino's 5V and GND pins.
3. The Arduino board was connected to the PC via USB.



4.0 Methodology

1. Setup the Arduino UNO R3
2. IDE code implementation
3. Python code implementation
4. Run the python code
5. Experiment the MPU6050
6. Read the angle in python terminal
7. Code snippet

4.1 Detail steps

1. Assemble the circuit as stated in the experimental setup.
2. Connect the arduino to the computer via usb cable, and upload the code to arduino IDE.
3. Close the arduino IDE and open python to run the python code.
4. Move the MPU6050 to test the value of the accelerometer and gyroscope output.
5. Observe the led lamp whether its on or not.

4.2 Programming codes

Arduino code

```
#include <Wire.h>

#include <Adafruit_MPU6050.h>

#include <Adafruit_Sensor.h>

Adafruit_MPU6050 mpu;

// --- DEFINE YOUR LED PIN HERE ---
```

```
const int ledPin = 7; // LED pin
void setup() {
Serial.begin(9600);
while (!Serial) delay(10); // wait for serial
if (!mpu.begin()) {
Serial.println("MPU6050 connection failed!");
while (1);
}
Serial.println("MPU6050 connected!");
// Configure accelerometer range (optional)
mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
mpu.setGyroRange(MPU6050_RANGE_500_DEG);
mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
pinMode(ledPin, OUTPUT);
digitalWrite(ledPin, LOW); // Start with LED off
}
void loop() {
// 1. Read motion data
sensors_event_t a, g, temp;
mpu.getEvent(&a, &g, &temp);
// 2. Send formatted data to Python (ax, ay, az)
Serial.print(a.acceleration.x); Serial.print(',');
Serial.print(a.acceleration.y); Serial.print(',');
```

```
Serial.println(a.acceleration.z);

// 3. Listen for commands from Python to control the LED

if (Serial.available() > 0) {

char cmd = Serial.read();

if (cmd == 'O') {

digitalWrite(ledPin, HIGH); // Turn LED ON

}

else if (cmd == 'F') {

digitalWrite(ledPin, LOW); // Turn LED OFF

}

}

delay(50); // Small delay for stability

}
```

Python code

```
import serial

import time

from collections import deque

import numpy as np

# -- Replace 'COMx' with your Arduino's serial port

SERIAL_PORT = 'COM6'

BAUD_RATE = 9600

# Buffers to store the last 40 data points (approx. 2 seconds of motion)
```

```
history_x = deque(maxlen=40)

history_y = deque(maxlen=40)

def is_circular_motion(x_data, y_data):

"""

Analyzes X and Y accelerometer data.

Returns True if the movement represents a circular motion.

"""

if len(x_data) < 10:

return False

x = np.array(x_data)

y = np.array(y_data)

# Step 1: Center the data

x_centered = x - np.mean(x)

y_centered = y - np.mean(y)

# Step 2: Amplitude check (ignore tiny jitters)

if np.std(x_centered) < 0.5 or np.std(y_centered) < 0.5:

return False

# Step 3: Circularity check (std ratio close to 1)

ratio = np.std(x_centered) / np.std(y_centered)

if ratio < 0.7 or ratio > 1.3:

return False

# Step 4: Continuous rotation check using cross product

cross_products = []
```

```
for i in range(len(x_centered) - 1):

x1, y1 = x_centered[i], y_centered[i]

x2, y2 = x_centered[i + 1], y_centered[i + 1]

cross = x1 * y2 - y1 * x2

cross_products.append(cross)

cross_products = np.array(cross_products)

clockwise_ratio = np.sum(cross_products > 0) / len(cross_products)

counter_ratio = np.sum(cross_products < 0) / len(cross_products)

# Step 5: Threshold for circle detection

if clockwise_ratio > 0.7 or counter_ratio > 0.7:

return True

return False

def main():

ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)

time.sleep(2) # Wait for Arduino to reset

print("System ready. Move the MPU6050 in a circle to toggle the LED...")

led_is_on = False

try:

while True:

raw = ser.readline().decode('utf-8', errors='ignore').strip()

if raw:

parts = raw.split(',')

if len(parts) >= 2:
```

```
try:
ax = float(parts[0])
ay = float(parts[1])
history_x.append(ax)
history_y.append(ay)
if len(history_x) == 40:
if is_circular_motion(history_x, history_y):
led_is_on = not led_is_on
if led_is_on:
print("Circle Detected! Toggling LED: ON")
ser.write(b'O')
else:
print("Circle Detected! Toggling LED: OFF")
ser.write(b'F')
history_x.clear()
history_y.clear()
except ValueError:
continue
except KeyboardInterrupt:
print("\nProgram terminated by user.")
finally:
ser.write(b'F')
ser.close()
```

```
print("Serial connection closed.")

if __name__ == "__main__":

main()
```

5.0 Data Collection

In this experiment, data collection was performed using an MPU6050 sensor connected to Arduino Uno. On the computer, a python program receives the serial data and updates it in real time. This setup allows for continuous streaming of hand motion data, which can then be used to observe motion patterns and understand the dynamic behavior of acceleration during different gestures.

6.0 Data Analysis

The accelerometer data collected from the MPU6050 sensor was analyzed using a combination of statistical and mathematical techniques to identify circular motion. A buffer of 40 consecutive X and Y axis readings was used to represent a short time window of movement. First, the mean values of both axes were calculated to determine the center of motion and remove any offset caused by sensor positioning. To further classify the motion, the standard deviation of the X and Y data was analyzed, and the ratio between them was used to evaluate symmetry. A ratio close to one indicates a balanced, circular motion, while a large deviation suggests linear or irregular movement. Additionally, the cross product between consecutive data points was computed to determine the direction of motion. By analyzing the consistency of rotation, the system checks whether the movement predominantly follows a single direction, either clockwise or counterclockwise. If more than 70% of the motion maintains the same rotational direction, the system classifies it as circular motion and triggers the LED response.

7.0 Results

This experiment successfully demonstrated real-time communication between the Arduino and Python environments with the MPU6050 accelerometer providing continuous raw acceleration data. The Arduino code was able to reliably capture the X, Y, and Z axis acceleration values and send them over serial communication without any glitches or data loss. Each packet of acceleration data reached the Python script in the expected comma-separated format indicating that the sensor and microcontroller were functioning properly.

In Python, the system accurately converted the raw acceleration readings to approximate physical units by scaling the data to a g value. The gravity offset applied to the Z axis allowed the program to isolate dynamic motion more effectively. Integrating these acceleration values produced a continuous velocity estimate across all three axes. While the integration was intentionally simple, it still produced clear changes in the estimated velocity in response to hand movement confirming that the MPU6050 readings were being processed correctly.

This experiment also achieved successful real-time trajectory plotting. The program dynamically updated the X and Y movement paths to produce a visual trace of the motion detected by the sensor. When the sensor is moved in different directions, the plotted lines shift accordingly, indicating that the integration logic and position estimation respond immediately to the accelerometer input. Despite the known numerical drift challenges in accelerometer-only systems, the short-term motion traces are accurate enough to reflect the general shape and direction of the hand movement.

Finally, limiting the buffer to the 100 most recent data points ensures smooth visualization and prevents memory issues, allowing the plots to be refreshed consistently throughout the test. Overall, the results confirm that the experiment successfully captured motion data, processed it, and visualized the movement in real time.

8.0 Discussion

The experiment effectively demonstrated simple hand gesture recognition using data from the MPU6050 sensor connected to an Arduino. The MPU6050 is the perfect sensor for identifying motion and orientation changes since it integrates gyroscope and accelerometer readings. We were able to record and transmit sensor data to a computer for additional processing by connecting the MPU6050 to an Arduino via I2C. We utilise Python on the computer to read and show this data in real-time by setting up serial connection. The Arduino code achieved basic gesture recognition, identifying predefined hand movements based on conditions based on threshold values in the accelerometer data. While Gesture 2 was identified with opposing circumstances on the X and Y axes, Gesture 1 was discovered when the accelerometer's X-axis data surpassed a threshold and the Y-axis was below it.

Although the experiment was successful in identifying basic gestures, there are a number of difficulties and restrictions. For example, it can be difficult to set suitable thresholds because different users may make gestures of different intensities, and small adjustments to the environment or sensor position may cause readings to change. Inconsistencies in gesture detection may result from this. Notwithstanding its drawbacks, the configuration laid the foundation for more complex gesture recognition systems by showcasing the viability of using accelerometer and gyroscope data for straightforward gesture-based applications.

9.0 Conclusion

In conclusion, an interactive motion detection system using the MPU6050 sensor has been successfully developed. The system is capable of detecting circular motion using real-time data processing in Python and controlling an LED via Arduino.

The project highlights the importance of signal processing and mathematical analysis in motion detection. It also demonstrates effective communication between hardware and software components

10.0 Recommendations

To improve the performance and reliability of the system, several enhancements can be considered. The detection accuracy can be increased by fine-tuning the threshold values used in variance, symmetry, and rotation checks to better suit different motion speeds and user behavior. Incorporating gyroscope data from the MPU6050 would provide more precise motion tracking and improve detection robustness. Additionally, implementing advanced filtering techniques can help reduce noise and external interference. The system could also be extended by adding graphical visualization of motion data for better analysis and debugging, as well as integrating machine learning algorithms for more complex gesture recognition.

11.0 References

- RFID RC522 Tutorial with Arduino
  https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/
- Security Access using MFRC522 RFID Reader with Arduino
  https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/

12. Acknowledgments

Special thanks to Dr. ZULKIFLI BIN ZAINAL ABIDIN and Dr. WAHJU SEDIONO for their guidance and support during this experiment.

Student's Declaration

Certificate of Originality and Authenticity

We hereby certify that we are collectively responsible for the work presented in this report. The content reflects our original work, except where specific references or acknowledgements have been made. No part of this report has been completed or submitted by individuals or sources not identified within.

Furthermore, we confirm that this report is the result of a collaborative effort, with contributions made by all group members. The extent of each individual's contribution is documented within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Name: AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI	Read	[/]
Matric Number: 2310663	Understand	[/]
Signature: SYAHMI	Agree	[/]

Name: ALYA SYAFINA BINTI ABDUL RAZAK	Read	[/]
Matric Number: 2218084	Understand	[/]
Signature: SYAF	Agree	[/]